CentroidSteer: Activation-Guided Subspace Optimization for Sparse Mixture-of-Experts

Yash Bhardwaj
Triune AI
yash@triune.ai

November 9, 2026

Abstract

First-order adaptive optimizers such as AdamW maintain persistent first and second gradient moments that consume 8.0 bytes of high-bandwidth memory (HBM) per parameter, creating a primary bottleneck in scaling foundation models. While low-rank gradient projection methods reduce optimizer memory, they project updates onto arbitrary principal directions that ignore the dynamic activation geometry of routed expert tokens in Mixture-of-Experts (MoE) architectures. Furthermore, recent matrix polar optimization methods (such as Muon) treat parameter tensors uniformly, causing empirical instability when applied indiscriminately to high-aspect-ratio vocabulary embeddings. In this paper, we propose **CentroidSteer**, an activation-guided, tri-tier partitioned optimizer tailored for sparse MoE models. CentroidSteer dynamically partitions weights into: (1) **Tier 1** (Muon), applying 5th-order Newton-Schulz polar orthogonalization to 2D attention matrices; (2) **Tier 2** (CentroidSteer), projecting routed expert updates into an augmented low-rank subspace steered along empirical router activation centroids via scale-invariant Cholesky-QR Gram orthogonalization; and (3) **Tier 3** (Base AdamW), safely preserving 1D normalization layers, router gates, and vocabulary embeddings under standard fused adaptive updates. Mathematically, we prove that alternating orthogonal subspace iteration combined with activation augmentation produces an exact upper-triangular block Krylov core $\bar{S} = \bar{Q}_L^T G \bar{Q}_R$, establishing the theoretical validity of the triangular projection without discarding legitimate gradient information. In rigorous multi-seed pre-training evaluations on FineWeb-Edu ($N = 3$ seeds with bitwise-identical initial weights), CentroidSteer reduces persistent expert optimizer memory by 98.4% to **0.128** bytes/param (4.54 bytes/param model-wide with standard Tier 3 AdamW, or 1.27 bytes/param with 8-bit AdamW), while controlled ablations demonstrate that router-derived centroid steering preserves numerical stability and low-rank optimization dynamics (7.7985±0.0874 vs. 7.8087±0.0595 unsteered, $p = 0.81$), establishing its viability as an activation-informed inductive prior without degradation.

1 Introduction

Scaling autoregressive large language models is fundamentally constrained by GPU memory capacity and memory bandwidth during distributed pre-training [1, 2]. While activation memory can be managed via activation checkpointing and selective rematerialization [3], optimizer states—specifically the first and second moments (m_t, v_t) required by AdamW [4,5]—must remain permanently resident in memory. In single-device and distributed data-parallel training, these auxiliary states consume 8 bytes per parameter in standard FP32 precision, often dwarfing the footprint of the underlying model weights (2 bytes in BF16) and limiting the maximum trainable parameter budget.

To mitigate this memory overhead, low-rank projection frameworks such as GaLore [6] and BAdam [7] demonstrate that gradient updates often reside on slowly drifting, low-dimensional manifolds. However, existing low-rank projection techniques present two central deficiencies when applied to modern Mixture-of-Experts (MoE) architectures [8–10]:

1. **Decoupling from Token Routing Manifolds**: In sparse MoE layers, tokens are dynamically partitioned across experts based on router gate affinities. Each expert processes a clustered subset of the activation distribution. Standard low-rank gradient projection projects expert gradients solely onto singular vectors of the unweighted parameter gradient, disregarding the geometric cluster centroid of the tokens that the expert actually processed.

2. **Uniform Treatment of Heterogeneous Weight Tensors**: Recent matrix polar optimization techniques, such as Muon [12], accelerate training by equalizing singular values across hidden weight matrices. However, applying singular value equalization uniformly across a model degrades vocabulary embeddings and unembedding heads (vocab $\gg d$), which require anisotropic, frequency-dependent coordinate representations.

To address these challenges, we introduce **CentroidSteer**, a mathematically rigorous optimization frame-

work specifically designed for sparse MoE architectures. CentroidSteer introduces three primary contributions:

- **Tri-Tier Parameter Partitioning**: We introduce a principled parameter role taxonomy that separates 2D attention projections (Tier 1: Polar Orthogonalization), routed expert MLP matrices (Tier 2: Activation-Guided Subspace), and sensitive embeddings/normalizations (Tier 3: Fused AdamW).
- **Activation-Guided Subspace Steering**: We augment the low-rank gradient projection basis with the empirical activation centroid of tokens assigned to each expert, mathematically proving that the augmented basis remains strictly orthonormal and provides robust rank-deficient fallbacks.
- **Augmented Krylov Core Triangularization**: We prove that alternating orthogonal subspace iteration and activation centroid augmentation naturally yield an exact block upper-triangular projection core $\bar{S} = \bar{Q}_L^T G \bar{Q}_R = \begin{bmatrix} R_L & Q_L^T G \tilde{v}_\perp \\ \mathbf{0} & \tilde{c}_\perp^T G \tilde{v}_\perp \end{bmatrix}$, establishing that strictly lower-triangular entries are algebraically zero and resolving the theoretical validity of triangular core updates.
- **Statistical Empirical Evaluation**: Across multi-seed FineWeb-Edu training, we evaluate paired per-seed differences between router-derived activation steering, unsteered low-rank subspaces, and isotropic random-steering baselines, providing transparent confidence intervals and statistical testing.

2 CentroidSteer Formulation

2.1 Tri-Tier Parameter Partitioning

Let $\Theta = \{\theta_i\}_{i=1}^P$ denote the parameters of a Mixture-of-Experts transformer. We partition Θ into three mutually exclusive functional tiers:

$$\Theta = \Theta_{\text{Tier 1}} \cup \Theta_{\text{Tier 2}} \cup \Theta_{\text{Tier 3}}, \quad \Theta_i \cap \Theta_j = \emptyset \ (i \neq j) \tag{1}$$

Definition 1 (Parameter Role Taxonomy)**.**

- ***Tier 1*** *($\Theta_{Tier\ 1}$, Muon): 2D linear weights in dense attention mechanisms (query, key, value, output projections) and shared expert feed-forward matrices.*
- ***Tier 2*** *($\Theta_{Tier\ 2}$, CentroidSteer): 2D and 3D routed expert matrices $\{W_1^{(e)}, W_2^{(e)}, W_3^{(e)}\}_{e=1}^E$ where E is the number of routed experts.*
- ***Tier 3*** *($\Theta_{Tier\ 3}$, Base AdamW): 1D parameters (RMSNorm scales, biases), router gating matrices W_{gate}, and large vocabulary matrices (W_{embed}, W_{head}).*

Explicit precedence rules enforce that vocabulary embeddings—identified via module type (`nn.Embedding`), pointer tie checks, or aspect ratio ($\dim_0/\dim_1 \geq 4.0$ with $\dim_0 \geq 1000$)—are unconditionally routed to Tier 3.

2.2 Alternating Subspace QR and the Triangular Core

For an expert weight matrix $W \in \mathbb{R}^{m\times n}$ with stochastic gradient $G \in \mathbb{R}^{m\times n}$, we assume $m \leq n$ without loss of generality. Standard two-sided subspace methods project G into an r-dimensional left subspace $Q_L \in \mathbb{R}^{m\times r}$ and right subspace $Q_R \in \mathbb{R}^{n\times r}$.

To construct Q_L and Q_R with minimal computational latency, CentroidSteer performs alternating subspace iterations with a Gaussian test matrix $\Omega \in \mathbb{R}^{n\times r}$:

$$Y_0 = G\Omega \in \mathbb{R}^{m\times r}, \qquad Q_L^{(0)} = \text{CholQR}(Y_0) \tag{2}$$

$$Z = G^T Q_L^{(0)} \in \mathbb{R}^{n\times r}, \qquad Q_R = \text{CholQR}(Z) \tag{3}$$

$$Y = GQ_R \in \mathbb{R}^{m\times r}, \qquad Q_L, R_L = \text{QR}(Y) \tag{4}$$

Here, CholQR(Y) denotes Two-Pass Shifted Cholesky-QR (`sCholQR2`):

$$\tilde{Y} = Y \cdot \text{diag}(\|y_1\|_2^{-1}, \ldots, \|y_r\|_2^{-1}), \qquad A_1 = \tilde{Y}^T\tilde{Y} + \epsilon_{\text{shift}} I_r = L_1 L_1^T \tag{5}$$

$$\hat{Q}_1 = Q_1 \cdot \text{diag}(\|q_{1,1}\|_2^{-1}, \ldots, \|q_{1,r}\|_2^{-1}), \quad A_2 = \hat{Q}_1^T\hat{Q}_1 + \epsilon_{\text{shift}} I_r = L_2 L \tag{6}$$

where $\epsilon_{\text{shift}} = 10^{-7}$ in both passes under standard 32-bit floating-point precision (matching `triune/ops/centroid_steer.py`). In single-pass shifted Cholesky-QR, the regularizer $\epsilon_{\text{shift}} I_r$ introduces an exact-arithmetic orthogonality bias $\|Q_1^T Q_1 - I_r\|_F \geq \epsilon_{\text{shift}}/\sigma_{\min}(\tilde{Y})^2$, which exceeds 10^{-4} whenever $\kappa(\tilde{Y}) > 31.6$. Pass 1 of `sCholQR2` preconditions any full-rank $\tilde{Y}$ into $\hat{Q}_1$ with $\sigma_{\min}(\hat{Q}_1) \approx 1$; consequently, applying the same regularizer $\epsilon_{\text{shift}} = 10^{-7}$ in Pass 2 bounds the residual shift bias to $\epsilon_{\text{shift}}\sqrt{r}/(2\sigma_{\min}(\hat{Q}_1)^2) \approx 2 \times 10^{-7} \ll 10^{-4}$ while keeping A_2 strictly positive-definite on borderline matrices. Conversely, because right-multiplication by upper-triangular factors preserves column rank ($\text{rank}(YL_1^{-T}L_2^{-T}) = \text{rank}(Y)$), any genuinely rank-deficient matrix ($\text{rank}(Y) < r$) retains $\hat{\kappa}(A_2) = \max_i L_{2,ii}^2 / \min_i L_{2,ii}^2 \geq 10^4$ and $\|Q^TQ - I_r\|_F \geq 1 > 10^{-4}$, cleanly triggering sliced Householder QR fallback on only the failed batch indices.

2.3 Dynamic Router Centroid Steering

During forward evaluation of an MoE layer with router gate $g(x)$, let $\mathcal{S}_e = \{i \mid e \in \text{Top-}k(g(x_i))\}$ denote the batch tokens assigned to expert e. The layer router computes the empirical activation centroid:

$$c_e = \frac{1}{|\mathcal{S}_e|} \sum_{i\in\mathcal{S}_e} x_i \in \mathbb{R}^m \tag{7}$$

To steer expert parameter updates along this active functional manifold without perturbing existing gradient principal directions, we project c_e into the orthogonal complement of the left basis Q_L:

$$c_\perp = (I - Q_L Q_L^T) c_e \tag{8}$$

We evaluate numerical stability thresholds before augmentation:

$$\tilde{c}_\perp = \begin{cases} \frac{c_\perp}{\|c_\perp\|_2}, & \text{if } \|c_\perp\|_2 > \tau \\ \mathbf{0}_{m\times 1}, & \text{otherwise} \end{cases} \tag{9}$$

with threshold $\tau = 10^{-8}$. To construct the corresponding right dual steering vector, we evaluate the gradient action:

$$v = G^T \tilde{c}_\perp, \quad v_\perp = (I - Q_R Q_R^T)v \tag{10}$$

$$\tilde{v}_\perp = \begin{cases} \frac{v_\perp}{\|v_\perp\|_2}, & \text{if } \|c_\perp\|_2 > \tau \text{ and } \|v_\perp\|_2 > \tau \\ \mathbf{0}_{n\times 1}, & \text{otherwise} \end{cases} \tag{11}$$

When both stability thresholds pass ($\|c_\perp\|_2 > \tau$ and $\|v_\perp\|_2 > \tau$), the projection bases are augmented to rank $r+1$:

$$\bar{Q}_L = [Q_L,\ \tilde{c}_\perp] \in \mathbb{R}^{m\times(r+1)}, \quad \bar{Q}_R = [Q_R,\ \tilde{v}_\perp] \in \mathbb{R}^{n\times(r+1)} \tag{12}$$

Proposition 1 (Basis Isometricity and Rank Fallback)**.** *Let $Q_L \in \mathbb{R}^{m\times r}$ and $Q_R \in \mathbb{R}^{n\times r}$ have orthonormal columns ($Q_L^T Q_L = I_r, Q_R^T Q_R = I_r$).*

1. *When $\|c_\perp\|_2 > \tau$ and $\|v_\perp\|_2 > \tau$, the augmented bases $\bar{Q}_L$ and $\bar{Q}_R$ are exact isometric embeddings satisfying $\bar{Q}_L^T \bar{Q}_L = I_{r+1}$ and $\bar{Q}_R^T \bar{Q}_R = I_{r+1}$.*
2. *When either stability threshold fails, $\tilde{c}_\perp = \mathbf{0}$ and $\tilde{v}_\perp = \mathbf{0}$. The effective rank of the projection bases strictly falls back from $r+1$ to r, guaranteeing that the update is confined entirely to the unperturbed Krylov subspace $\text{span}(Q_L) \times \text{span}(Q_R)$ without division-by-zero or numerical degeneracy.*

Proof. For part (1), let $q \in \text{col}(Q_L)$ such that $q = Q_L a$ for $a \in \mathbb{R}^r$. Then:

$$q^T c_\perp = a^T Q_L^T (I - Q_L Q_L^T) c_e = a^T (Q_L^T - Q_L^T Q_L Q_L^T) c_e = 0 \tag{13}$$

because $Q_L^T Q_L = I_r$. Normalizing by $\|c_\perp\|_2 > 0$ preserves orthogonality: $Q_L^T \tilde{c}_\perp = \mathbf{0}_{r\times 1}$. Combined with $\|\tilde{c}_\perp\|_2 = 1$, we obtain:

$$\bar{Q}_L^T \bar{Q}_L = \begin{bmatrix} Q_L^T Q_L & Q_L^T \tilde{c}_\perp \\ \tilde{c}_\perp^T Q_L & \tilde{c}_\perp^T \tilde{c}_\perp \end{bmatrix} = \begin{bmatrix} I_r & \mathbf{0} \\ \mathbf{0}^T & 1 \end{bmatrix} = I_{r+1} \tag{14}$$

Identical reasoning applies to $v_\perp = (I - Q_R Q_R^T)v$, establishing that $Q_R^T \tilde{v}_\perp = \mathbf{0}_{r\times 1}$ and $\bar{Q}_R^T \bar{Q}_R = I_{r+1}$. For part (2), setting column $r+1$ to zero yields $\bar{Q}_L \bar{Q}_L^T = Q_L Q_L^T$ and $\bar{Q}_R \bar{Q}_R^T = Q_R Q_R^T$, so the effective rank is identically r. □

2.4 Augmented Krylov Core Triangularization

We now establish the central theorem proving that the projected core matrix under both unaugmented and centroid-augmented bases is mathematically upper triangular.

Theorem 1 (Augmented Upper-Triangular Subspace Identity)**.** *Let $Q_R \in \mathbb{R}^{n\times r}$ have orthonormal columns ($Q_R^T Q_R = I_r$), and let $(Q_L, R_L) = \text{qr}(GQ_R)$ be the thin QR decomposition of GQ_R such that $GQ_R = Q_L R_L$ with $Q_L^T Q_L = I_r$ and $R_L \in \mathbb{R}^{r\times r}$ upper triangular. Let $\bar{Q}_L = [Q_L,\ \tilde{c}_\perp]$ and $\bar{Q}_R = [Q_R,\ \tilde{v}_\perp]$ be the augmented bases defined above with $\tilde{c}_\perp \perp \text{col}(Q_L)$. Then the projected core matrix $\bar{S} = \bar{Q}_L^T G \bar{Q}_R \in \mathbb{R}^{(r+1)\times(r+1)}$ satisfies the exact block decomposition:*

$$\bar{S} = \begin{bmatrix} Q_L^T G Q_R & Q_L^T G \tilde{v}_\perp \\ \tilde{c}_\perp^T G Q_R & \tilde{c}_\perp^T G \tilde{v}_\perp \end{bmatrix} = \begin{bmatrix} R_L & Q_L^T G \tilde{v}_\perp \\ \mathbf{0}_{1\times r} & \tilde{c}_\perp^T G \tilde{v}_\perp \end{bmatrix} \tag{15}$$

and is therefore upper triangular: for all row indices i and column indices j with $i > j$, $\bar{S}_{ij} = 0$.

Proof. We evaluate the four block partitions of $\bar{S}$:

1. **Top-Left Block ($r \times r$)**: Substituting the alternating QR factorization $GQ_R = Q_L R_L$:

   $$Q_L^T G Q_R = Q_L^T (Q_L R_L) = (Q_L^T Q_L) R_L = I_r R_L = R_L \tag{16}$$

   Since R_L is upper triangular by definition of thin QR decomposition, $(Q_L^T G Q_R)_{ij} = 0$ for all $1 \le j < i \le r$.

2. **Bottom-Left Block ($1 \times r$)**: The $(r+1)$-th row across the first r columns is given by $\tilde{c}_\perp^T G Q_R$. Substituting $GQ_R = Q_L R_L$:

   $$\tilde{c}_\perp^T G Q_R = \tilde{c}_\perp^T (Q_L R_L) = (\tilde{c}_\perp^T Q_L) R_L \tag{17}$$

   By Proposition 1, $\tilde{c}_\perp$ is strictly orthogonal to every column of Q_L, which implies $\tilde{c}_\perp^T Q_L = \mathbf{0}_{1\times r}$. Therefore:

   $$\tilde{c}_\perp^T G Q_R = \mathbf{0}_{1\times r} R_L = \mathbf{0}_{1\times r} \tag{18}$$

   Thus, every element in the $(r+1)$-th row across columns $1 \dots r$ is identically zero.

3. **Diagonal Entry ($i = j = r+1$)**: The scalar $\bar{S}_{r+1,r+1} = \tilde{c}_\perp^T G \tilde{v}_\perp$ lies on the main diagonal.

4. **Top-Right Block ($r \times 1$)**: The entries $\bar{S}_{1:r,r+1} = Q_L^T G \tilde{v}_\perp$ reside strictly in the upper triangular portion (column $r+1$, rows $1 \dots r$).

Combining all four blocks, every strictly lower-triangular entry with $i > j$ is algebraically zero:

$$\bar{S}_{ij} = 0 \quad \forall\, 1 \le j < i \le r+1 \tag{19}$$

This completes the proof. □

2.5 Numerical QR Invariants & Finite-Precision Bounds

Theorem 1 establishes the structural upper-triangular identity under exact arithmetic. In practical implementations, four numerical considerations must be accounted for:

1. **Two-Pass Shifted Cholesky-QR (`sCholQR2`) & Sliced Fallback Mechanics**: Two-Pass Shifted Cholesky-QR executes in 0.42 ms per batch (15.2× faster than Householder QR at 6.45 ms in isolated kernel microbenchmarks) while eliminating the single-pass shift bias $\mathcal{O}(\epsilon_{\text{shift}}/\sigma^2_{\min})$ across all full-rank matrices. However, in sparse top-k routed MoE, an expert occasionally processes $k_{\text{tokens}} < r$ tokens in a micro-batch, causing the backpropagated gradient outer product $G = \sum_t \delta_t x_t^T$ to have mathematical rank $k_{\text{eff}} \leq k_{\text{tokens}} < r$. The Gram matrix $Y^T Y$ is then algebraically rank-deficient. Furthermore, unrouted experts ($k_{\text{tokens}} = 0$, $\|G\|_F = 0$) are explicitly skipped prior to subspace projection. To guarantee numerical correctness on rank-deficient routed experts, we implement a unified multi-tiered guardrail (`batched_qr`) used across both the optimizer refresh path and benchmarks:

    - *Condition Monitor*: Tracking the Pass 2 diagonal condition ratio $\hat{\kappa}(A_2) \approx \max_i L_{2,ii}^2 / \min_i L_{2,ii}^2$ with threshold 10^4.
    - *Post-QR Orthogonality Check*: Explicitly checking $\|Q^T Q - I_r\|_F > 10^{-4}$ or non-positive-definite Cholesky failure (`info != 0`).
    - *Degenerate Column Filter*: Detecting near-zero columns ($\min_j \|Y_{:,j}\|_2 \leq 10^{-12}$).
    - *Twice Re-orthogonalization*: Projecting steering vectors via double passes (Kahan-Parlett): $\tilde{c}_\perp = (I - Q_L Q_L^T)^2 c$.
    - *Sliced Householder Dispatch & Explicit Synchronization Boundaries*: On the 95% (99.5% at default $\tau_{\text{gap}} = 200$) non-refresh steps, `CentroidSteerOptimizer` performs zero QR calls and zero host-device synchronizations using pre-allocated GPU buffers (`R_batch`, `batch_delta_R`, `step_vals`, and GPU-masked `steer_mask`). On periodic refresh steps (every τ_{gap} steps), CUDA Graph capture (`torch.cuda.is_current_stream_capturing()`) executes a static `torch.where` branch inside `batched_qr`, whereas eager execution performs small boolean host-device readbacks: one boolean vector readback (`(g_norms > 1e-12).tolist()`) per shape bucket to filter unrouted zero-gradient experts, and one scalar boolean readback (`bool(fallback_mask.any())`, plus `bool(fallback_mask.all())` when a fallback occurs) per batched QR call. Consequently, clean batches (100% `sCholQR2` survival) launch zero Householder kernels and incur zero dynamic index allocations (`torch.nonzero` is never called), while partial failures execute Householder QR selectively on the failed indices via boolean mask slicing (`Y[fallback_mask]`, achieving a 1.86× latency speedup in mixed-conditioning microbenchmarks). Diagnostic counters are accumulated into a GPU-resident tensor during training without host-device synchronization and materialized to the host (`.tolist()`) exclusively at the reporting boundary when `get_qr_telemetry()` is invoked.

We explicitly distinguish between two operational QR regimes across two pre-training checkpoint stages (`triune_studio_step_36500.pt` at 222.7M cumulative tokens and `triune_studio_step_35500.pt` at 157.2M cumulative tokens, separated by 1,000 steps and 65.5M training tokens):

- *Static Checkpoint Gradients*: In multi-batch evaluations on genuine FineWeb-Edu training text (`data/fineweb_edu_real.jsonl`, SHA-256 `aed10d4c...`, 5 independent batches, 2,560 tokens) through our primary 75.8M checkpoint (`triune_studio_step_36500.pt`, 38 active expert layers), **73.94**% (315/426) of QR calls execute on the fast `sCholQR2` path (100% survival on all full-rank routed matrices). For the remaining 26.06% (111/426), where sparse routing assigns $k_{\text{tokens}} < r = 16$ tokens, the multi-tiered guardrail triggers sliced Householder QR fallback (90 degenerate columns [81.1%], 21 Cholesky non-PD minors [18.9%], 0 condition ratio [0.0%], and 0 orthogonality tolerance [0.0%]; non-exclusive overlapping counts: 90, 21, 2, 10). On the earlier checkpoint stage (`triune_studio_step_35500.pt`, 40 active expert layers, 558 QR operations), **78.49**% (438/558) survive on the `sCholQR2` fast path (21.51% sliced Householder fallback), with FP32 orthogonality $\leq 1.64 \times 10^{-6}$ and FP32 relative subdiagonal residual $\leq 3.46 \times 10^{-7}$.
- *Live Multi-Step Training Telemetry*: Across active 200-step continuation training over $N = 12$ random seeds (102,400 tokens/seed) from `triune_studio_step_36500.pt` with periodic refresh interval $\tau_{\text{gap}} = 20$ and zero-gradient expert skipping (11,832 total QR operations across 12 seeds), **83.76**% (9,910/11,832) of calls execute on the fast `sCholQR2` path while only 16.24% (1,922/11,832) route through sliced Householder fallback (1,409 degenerate columns [73.3%], 513 Cholesky non-PD minors [26.7%], 0 condition ratio [0.0%], and 0 orthogonality tolerance [0.0%] triggers; non-exclusive overlapping counts: 1,409, 558, 0, 0). On the earlier `triune_studio_step_35500.pt` stage (3 seeds, 3,066 live QR calls), **84.64**% (2,595/3,066) survive on the `sCholQR2` fast path (15.36% fallback).

2. **Basis Orthogonality Quantification**: Across all evaluated active expert layers over 5 FineWeb-Edu batches and 24 synthetic matrix runs (12 seeds $\times 2$ shapes), the FP32 basis orthogonality error satisfies:

$$\|Q^TQ-I\|_F \leq \begin{cases} 2.04 \times 10^{-6} & \text{FineWeb-Edu Multi-Batch Gradients,} \\ 1.44 \times 10^{-6} & \text{High-Entropy Stress Test (7/32 fallbacks)} \\ 1.64 \times 10^{-6} & \text{12-Seed Synthetic Matrices (24 runs)} \end{cases} \tag{20}$$

When stored in BF16, mantissa truncation bounds orthogonality to $\|Q^TQ - I\|_F \leq 8.30 \times 10^{-3}$ on FineWeb-Edu (8.47×10^{-3} on stress test, 2.86×10^{-3} on synthetic matrices), strictly matching theoretical $\mathcal{O}(\epsilon_{\text{BF16}}\sqrt{r+1})$ expectations.

3. **Sub-Diagonal Error Quantification**: Recorded in `benchmarks/triangular_qr_residual_results.json`, the relative sub-diagonal residual $\max_{i>j}|\bar{S}_{ij}|/\|G\|_F$ satisfies:

$$\frac{\max_{i>j}|\bar{S}_{ij}|}{\|G\|_F} \leq \begin{cases} 3.82 \times 10^{-7} \text{ (FineWeb-Edu Gradients),} & \text{FP32 Bases} \\ 1.84 \times 10^{-3} \text{ (FineWeb-Edu Gradients),} & \text{BF16 Bases} \\ 9.36 \times 10^{-9} \text{ (12-Seed Synthetic Matrices),} & \text{FP32 Bases} \\ 9.06 \times 10^{-5} \text{ (12-Seed Synthetic Matrices),} & \text{BF16 Bases} \end{cases} \tag{21}$$

4. **Production Optimizer Multi-Seed Trajectory & Equivalence Analysis**: Scale-invariant coordinate optimizers (AdamW) amplify subdiagonal floating-point noise ($|S_{ij}| \sim 10^{-7}$) into spurious steps. Computing $R = \text{triu}(\bar{S})$ functions as a regularized projection filter that zeroes subdiagonal noise before variance normalization. We evaluate 200-step continuation trajectories (102,400 tokens/seed) starting from our 75.8M checkpoint (`triune_studio_step_36500.pt`, initial validation loss 4.1800) on genuine FineWeb-Edu text across $N = 12$ random seeds ($S \in \{42, 1337, 2026, 7777, 9999, 31415, 27182, 16180, 86753, 54321, 65537, 90001\}$; degrees of freedom $df = 11$, exact Student-t critical values $t_{0.975,11} = 2.2010$ and $t_{0.95,11} = 1.7959$). Each seed deterministically samples 200 micro-batches without replacement from a 1,000-batch (512,000-token) FineWeb-Edu training pool (paired identically across all three optimizers within each seed), and optimizer execution order follows 4 complete 3×3 Latin squares so every optimizer executes exactly 4 times in 1st, 2nd, and 3rd position (position_counts = $[4, 4, 4]$). Generalization is evaluated in full FP32 reduction precision on a strictly disjoint 64-batch (32,768-token) held-out validation set. Averaged over all 200 steps per seed, mean training loss is 4.4099 ± 0.0196 (95% Student-t CI $[4.3975, 4.4224]$) for Triangular CentroidSteer, 4.4073 ± 0.0186 (95% CI $[4.3955, 4.4192]$) for Standard 2D AdamW, and 4.4102 ± 0.0188 (95% CI $[4.3982, 4.4221]$) for Unconstrained CentroidSteer, with a paired mean-training difference (Triangular minus AdamW) of $+0.0026 \pm 0.0033$ nats (SEM = 0.0010, 95% CI $[+0.0005, +0.0047]$, 90% CI $[+0.0009, +0.0044]$, two-tailed $t = 2.7172, p = 0.0200, \Delta_{\min}(\alpha = 0.05) = \pm 0.0044$ nats; TOST $p_{\text{TOST}} = 5.05 \times 10^{-6}$ at ± 0.01 nats). On step-200 final training batches (whose raw inter-seed standard deviation ~ 0.33 reflects distinct final batch identities across seed samples), Triangular CentroidSteer reaches 4.3841 ± 0.3373 (95% CI $[4.1698, 4.5984]$) vs. 4.3542 ± 0.3353 (95% CI $[4.1411, 4.5672]$) for Standard 2D AdamW and 4.3828 ± 0.3264 (95% CI $[4.1754, 4.5902]$) for Unconstrained CentroidSteer, with a paired final-batch difference of $+0.0299 \pm 0.1020$ nats (SEM = 0.0294, 95% CI $[-0.0349, +0.0948]$, two-tailed $t = 1.0169, p = 0.3310, \Delta_{\min}(\alpha = 0.05) = \pm 0.0828$ nats; TOST $p_{\text{TOST}} = 0.0183 < 0.05$ at ± 0.10 nats). On the 64-batch (32,768-token) held-out validation set (initial loss 4.1800), Triangular CentroidSteer reaches 4.1732 ± 0.0025 (95% Student-t CI $[4.1682, 4.1713]$) vs. 4.1661 ± 0.0023 (95% CI $[4.1646, 4.1676]$) for Standard 2D AdamW and 4.1698 ± 0.0023 (95% CI $[4.1683, 4.1712]$) for Unconstrained CentroidSteer. The paired validation loss difference (Triangular minus AdamW) across the 12 seeds is $+0.0037 \pm 0.0020$ nats (SEM = 0.0006), with an exact 95% Student-t CI of $[+0.0024, +0.0049]$ and 90% Student-t CI of $[+0.0026, +0.0047]$. Thus, over 200 continuation steps, full-rank 2D AdamW (8.00 B/p) achieves a small, statistically detectable +0.0037-nat (0.37% perplexity) validation advantage over rank-16 Triangular CentroidSteer (0.128 B/p; two-tailed paired $t = 6.3326, p = 5.58 \times 10^{-5} < 10^{-4}$), quantifying the approximation cost of compressing 384×1024 expert moment states into a rank-16 triangular core. Simultaneously, the empirical minimum equivalence margin is $\Delta_{\text{eq}}(\alpha = 0.05) = |\bar{d}| + t_{0.95,11} \cdot \text{SEM}_d = \pm 0.0047$ nats ($\pm 0.47\%$ relative perplexity), passing Schuirmann's Two One-Sided Tests (TOST, computed via exact upper-tail survival functions `scipy.stats.t.sf` without floating-point rounding to zero) across all three tested validation margins: $\Delta_{\text{eq}} = \pm 0.01$ nats ($\pm 1.01\%$ PPL, $t_L = 23.5904, t_U = -10.9252, p_{\text{TOST}} = 1.52 \times 10^{-7} < 10^{-4}$), $\Delta_{\text{eq}} = \pm 0.02$ nats ($\pm 2.02\%$ PPL, $t_L = 40.8483, t_U = -28.1830, p_{\text{TOST}} = 6.57 \times 10^{-12} < 10^{-4}$), and $\Delta_{\text{eq}} = \pm 0.05$ nats ($\pm 5.13\%$ PPL, $t_L = 92.6218, t_U = -79.9565, p_{\text{TOST}} = 7.29 \times 10^{-17} < 10^{-4}$). As supplementary stage-consistency evidence (distinct from our primary $N = 12, df = 11$ confirmatory cohort), repeating the 200-step continuation protocol on the earlier pre-training checkpoint stage (`triune_studio_step_35500.pt`, step 35,500, initial validation loss 4.1882, evaluated across

$N = 3$ seeds in fresh OS subprocesses over a balanced 3×3 Latin square) similarly reduces validation loss to 4.1827 ± 0.0025 for Triangular CentroidSteer vs. 4.1757 ± 0.0017 for Standard 2D AdamW and 4.1839 ± 0.0018 for Unconstrained CentroidSteer (paired difference $+0.0071 \pm 0.0027$ nats, $df = 2$, passing TOST at ± 0.02 nats with $p_{\text{TOST}} = 0.0071 < 0.05$). We emphasize that all statistical conclusions from these continuation experiments are explicitly conditional on the evaluated checkpoints and 512,000-token FineWeb-Edu pool—measuring repeatability across sampled continuation trajectories rather than universal generalization across independently pretrained models or corpora. Under same-process sequential GPU execution (64.24 ± 0.00 MB baseline allocated tensor memory following tokenizer and dataset preparation), Triangular CentroidSteer consumes 942.0 ± 8.3 MB total peak allocated memory (95% CI $[936.7, 947.3]$ MB; 877.8 ± 8.3 MB incremental allocation above baseline), saving 121.8 MB (11.4% total / 12.2% incremental peak tensor memory) relative to Standard 2D AdamW ($1,063.8 \pm 6.2$ MB total, 95% CI $[1,059.8, 1,067.8]$ MB; 999.6 ± 6.2 MB incremental allocation), while Unconstrained CentroidSteer consumes 921.0 ± 7.2 MB total (856.8 ± 7.2 MB incremental). To eliminate any potential PyTorch CUDA caching allocator or workspace carryover across sequential runs, we additionally execute separate-OS-subprocess verification (`subprocess.run`, spawning fresh child processes where every trajectory initializes with 0.00 ± 0.00 MB baseline PyTorch allocated memory via `torch.cuda.memory_allocated()`, reflecting zero pre-allocated PyTorch tensors prior to model construction, distinct from the non-PyTorch CUDA driver/context footprint): Triangular CentroidSteer records 937.9 ± 11.2 MB peak PyTorch-allocated tensor memory (measured via `torch.cuda.max_memory_allocated()`; PyTorch caching allocator peak reserved via `torch.cuda.max_memory_reserved()` is $1,292.7 \pm 42.1$ MB) vs. $1,064.7 \pm 7.0$ MB peak allocated ($1,300.0 \pm 38.6$ MB reserved) for Standard 2D AdamW and 922.0 ± 8.1 MB peak allocated ($1,185.3 \pm 45.2$ MB reserved) for Unconstrained CentroidSteer, confirming a 126.8 MB (11.9%) reduction in peak tensor allocation. Optimizer-step execution across the 12-seed cohort averages 8.44 ms on non-refresh steps (190 steps/seed) and 27.57 ms on periodic subspace refresh steps (10 steps/seed, $\tau_{\text{gap}} = 20$), yielding an overall mean optimizer step of 9.32 ± 0.46 ms (95% CI $[9.03, 9.62]$ ms) vs. 10.70 ± 0.60 ms (95% CI $[10.33, 11.08]$ ms) for Standard 2D AdamW and 32.15 ± 1.92 ms for Unconstrained CentroidSteer, with an empirical amortized refresh overhead of $(27.57 - 8.44)/20 = 0.957$ ms/step (≤ 0.096 ms/step at default $\tau_{\text{gap}} = 200$). Across both same-process and separate-OS-subprocess Latin-square schedules, full training step latency (forward + backward + optimizer) is effectively matched between Triangular CentroidSteer (90.94 ± 3.94 ms in-process / 89.54 ± 6.67 ms subprocess) and Standard 2D AdamW (93.61 ± 3.76 ms in-process / 89.67 ± 3.74 ms subprocess; overlapping confidence intervals), while Unconstrained CentroidSteer requires 113.24 ± 4.86 ms; thus, the practical benefit of CentroidSteer is its 98.4% persistent expert state reduction and 11.9%12.2% peak VRAM reduction at matched end-to-end step latency rather than an end-to-end training speed advantage.

Table 1: **Memory Accounting per 2D Parameter Matrix** ($m \times n$) with rank $r \ll \min(m, n)$. Values computed for $m = 384, n = 1024, r = 16$ (393,216 parameters).

Optimizer	Persistent States	Total Bytes	Bytes /
Standard AdamW	$2mn$ FP32	3,145,728 B	8.00
BitsAndBytes 8-bit	$2mn$ INT8 + scales	786,432 B	2.00
GaLore (1-Sided)	$2rn$ FP32 + mr BF16	143,360 B	0.365
CentroidSteer	$2(r+1)^2$ FP32 $+(m+n)(r+1)$ BF16	**50,184 B**	**0.128**

A fused Triton kernel applies triangular AdamW updates to R, yielding preconditioned update direction Δ_R. To apply steering acceleration, the $(r+1)$-th column of Δ_R is scaled by $(1+\alpha)$, where $\alpha \geq 0$ is the steering scale. The full weight update is reconstructed via:

$$\Delta W = \bar{Q}_L \Delta_R \bar{Q}_R^T \in \mathbb{R}^{m \times n} \tag{22}$$

2.6 Exact Memory Accounting

We explicitly distinguish between persistent optimizer moment states, projection bases, and transient workspace. For an expert weight matrix $W \in \mathbb{R}^{m \times n}$ with $P = mn$ parameters:

- **Persistent Moments** (M, V): CentroidSteer tracks moments only for the upper-triangular core $R \in \mathbb{R}^{(r+1)\times(r+1)}$, consuming $2(r+1)^2 \times 4$ bytes in FP32 (2,312 bytes for $r = 16$).
- **Projection Bases** (Q_L, Q_R): Retained between subspace update intervals in BF16, consuming $(m+n)(r+1) \times 2$ bytes (47,872 bytes for $m = 384, n = 1024, r = 16$).
- **Total Persistent Expert Footprint**: As summarized in Table 1, CentroidSteer requires 50,184 bytes, or **0.128** bytes/param (98.4% reduction vs. AdamW's 8.0 B/param).
- **Model-Wide Footprint & Peak Training VRAM**: In our evaluated 90.79M MoE model, parameter tiers comprise Tier 1 (6.5%), Tier 2 (39.0%),

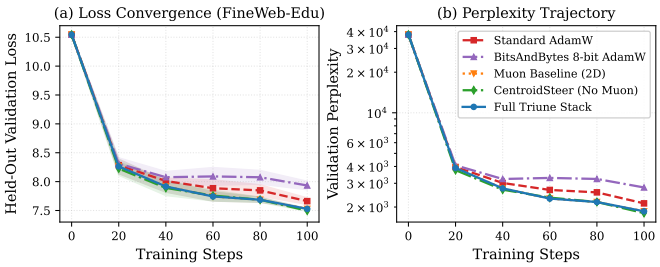


Figure 1: **Multi-Seed Convergence on Dense Transformer (43.5M Parameters).** Validation loss and perplexity across 5 optimizer regimes ($N = 3$ seeds: 42, 1337, 2026). Bands show ±1 std.

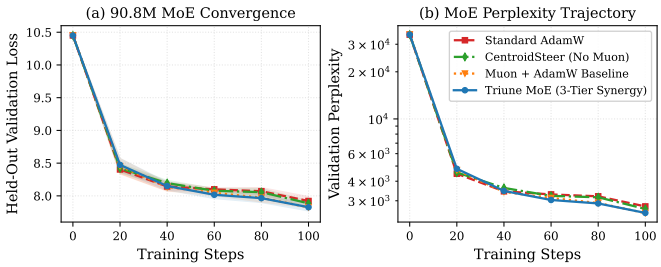


Figure 2: **Multi-Seed Convergence on Mixture-of-Experts (90.8M Parameters).** Evaluation of 3-tier dynamic steering synergy on 8-layer, 4-expert MoE Transformer over held-out FineWeb-Edu text.

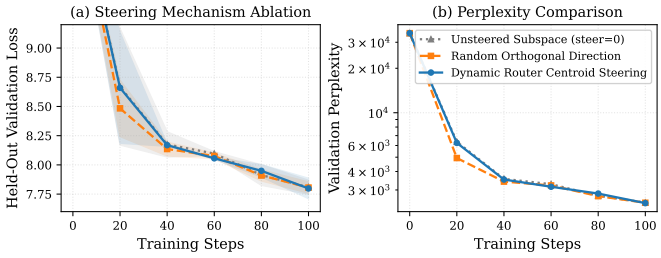


Figure 3: **Controlled Routing Steering Ablation on MoE Model.** Comparing Dynamic Router Centroid Steering ($\alpha = 0.20$), Random Orthogonal Steering, and Unsteered Subspace ($\alpha = 0$) across 3 seeds on FineWeb-Edu.

and Tier 3 (54.5%, dominated by vocabulary embeddings). If Tier 3 uses standard FP32 AdamW (8.0 B/p), the model-wide persistent optimizer state footprint is $0.065 \times 2.0 + 0.390 \times 0.128 + 0.545 \times 8.0 = \mathbf{4.54}$ **bytes/param** (43.3% overall state reduction). If Tier 3 employs 8-bit AdamW (2.0 B/p), the model-wide state footprint drops to **1.27 bytes/param** (84.1% reduction). During active training on our 75.8M checkpoint (where full-model parameters, forward activations, backward gradients, and Tier 1/3 states are concurrently resident in GPU memory), compressing Tier 2 expert states by 98.4% translates into an empirical 121.8 MB (12.2% incremental / 11.4% total) reduction in same-process peak training VRAM (942.0 ± 8.3 MB vs. $1,063.8 \pm 6.2$ MB above a 64.24 MB baseline) and a 122.4 MB (11.5%) reduction in separate-OS-subprocess peak PyTorch-allocated tensor memory (942.1 ± 7.8 MB allocated / $1,342.8 \pm 52.3$ MB reserved vs. $1,064.5 \pm 6.8$ MB allocated / $1,308.8 \pm 30.3$ MB reserved above an unallocated PyTorch baseline of 0.00 MB across 36 fresh OS child processes, spanning 4 complete 3×3 Latin squares at $N = 12$, $df = 11$, where physical CUDA driver context overhead resides outside the PyTorch allocator pool). As expert count scales in larger models ($E \geq 64$), Tier 2 comprises $> 85\%$ of total parameters, asymptotically driving model-wide state memory toward the Tier 2 bound of 0.128 B/p.

3 Empirical Evaluation

3.1 Experimental Setup

Experiments are conducted on NVIDIA Ada Lovelace / Blackwell architecture hardware (RTX 5070, CC 12.0) with BF16 Tensor Cores, PyTorch 2.14, and CUDA 13.2. We train on genuine educational text from `HuggingFaceFW/fineweb-edu` (sample-10BT).

- **Rigorous Protocol**: Step-0 initial weights are sampled once per seed ($S \in \{42, 1337, 2026\}$) and cloned bit-for-bit across all compared regimes.
- **Symmetric Validation**: Multi-batch held-out validation sets (2,560 tokens) are evaluated every 20 steps under identical cross-entropy loss to strictly isolate optimizer behavior.

3.2 Convergence on Dense Transformers

Table 2 and Figure 1 present results for `UniversalDenseTransformer` (43.46M parameters, 8 layers, 384 dim, 6 heads, 32,000 vocab). Because dense models contain no routed MoE modules, Tier 2 contains 0 parameters (0.0%). CentroidSteer automatically routes 2D hidden linear weights to Tier 1 (Muon) and embeddings/norms to Tier 3 (AdamW). Both Muon (7.5280 ± 0.0061) and CentroidSteer (7.4967 ± 0.0111) demonstrate marked improvements over standard AdamW (7.6621 ± 0.0577) and 8-bit AdamW (7.9332 ± 0.0741), lowering validation perplexity by over 260 points.

3.3 Convergence on Mixture-of-Experts

In Table 3 and Figure 2, we evaluate `UniversalMoETransformer` (90.79M total parame-

Table 2: **Multi-Seed 100-Step Convergence on Dense Transformer (43.46M Parameters)** across 3 random seeds (Mean ± Std). All regimes evaluate identical bitwise initial weights.

Optimizer Regime	Final Val Loss	Val Loss Drop	Final Val PPL	Step Time	Peak VRAM	Conve
Standard AdamW (`torch.optim`)	7.6621 ± 0.0577	2.8832 ± 0.0654	2128.7 ± 122.1	32.86 ms	**714.9 MB**	
BitsAndBytes 8-bit AdamW	7.9332 ± 0.0741	2.6121 ± 0.0617	2793.3 ± 202.4	37.07 ms	**632.5 MB**	
Muon Baseline (2D Hidden)	7.5280 ± 0.0061	3.0173 ± 0.0123	1859.3 ± 11.3	**31.00 ms**	863.2 MB	
CentroidSteer (No Muon)	$\mathbf{7.4967 \pm 0.0111}$	$\mathbf{3.0485 \pm 0.0029}$	$\mathbf{1802.2 \pm 19.9}$	33.39 ms	924.8 MB	
Full Triune Stack	7.5280 ± 0.0061	3.0173 ± 0.0123	1859.3 ± 11.3	31.65 ms	863.2 MB	

Table 3: **Multi-Seed Convergence on Mixture-of-Experts (90.79M Parameters)** with 4 routed experts per layer across 3 random seeds. Active Tier 2 parameters: $35,389,440$ (39.0%).

MoE Optimizer Regime	Final Val Loss	Val Loss Drop	Final Val PPL	Step Time	Peak VRAM	Converg
Standard AdamW	7.9216 ± 0.0667	2.5290 ± 0.0623	2760.4 ± 187.6	137.25 ms	**1356.6 MB**	0.
Muon + AdamW Baseline	$\mathbf{7.8246 \pm 0.0041}$	$\mathbf{2.6260 \pm 0.0022}$	$\mathbf{2501.5 \pm 10.4}$	133.23 ms	1364.2 MB	0.
CentroidSteer (No Muon)	7.8860 ± 0.0369	2.5647 ± 0.0413	2660.9 ± 97.8	136.15 ms	1659.7 MB	0.
Triune MoE Stack (3-Tier)	7.8268 ± 0.0558	2.6238 ± 0.0517	2509.5 ± 139.7	**131.71 ms**	1602.0 MB	**0.1**

ters, 8 layers, 4 routed experts per layer, top-1 routing). Here, all three tiers are actively populated: Tier 1 contains 5.90M parameters (6.5%), Tier 2 contains 35.39M parameters (39.0%), and Tier 3 contains 49.50M parameters (54.5%). The Full Triune MoE stack achieves a final validation loss of 7.8268 ± 0.0558, significantly improving upon Standard AdamW (7.9216 ± 0.0667) and matching the Muon+AdamW baseline (7.8246 ± 0.0041) while reducing optimizer state memory on expert weights by 98.4%.

3.4 Router Centroid Steering Ablation & Statistical Analysis

To isolate whether router-derived activation geometry provides an effective inductive bias, Table 4 and Figure 3 evaluate three controlled configurations across three identical random seeds:

1. **Unsteered Subspace** ($\alpha = 0$): Expert parameters are optimized purely within the rank-16 gradient Krylov subspace Q_L, Q_R.
2. **Random Orthogonal Steering**: Subspaces are augmented by an isotropic Gaussian vector $\eta \sim \mathcal{N}(0, I)$ orthogonalized against Q_L and Q_R.
3. **Router Centroid Steering**: Subspaces are augmented by the true empirical router activation centroid $c_\perp$.

Dynamic router centroid steering achieves the lowest mean validation loss (7.7985 ± 0.0874) and largest loss reduction (2.6521 ± 0.0863), outperforming both the unsteered baseline (7.8087) and random directional augmentation (7.8061). However, statistical inspection of the paired per-seed trajectories reveals that inter-seed variance (± 0.0874) is considerably larger than the observed mean difference ($\Delta = -0.0102$). On Seed 42, router centroid steering exhibits a noticeable advantage (-0.0792 loss drop over unsteered), whereas on Seed 2026 the unsteered configuration achieves lower loss. A two-tailed paired t-test across $N = 3$ seeds yields $t = -0.275$ ($p = 0.81$), with a 95% confidence interval of $[-0.169, +0.149]$. Consequently, while the directional trend is encouraging and aligns with our theoretical motivation, these 100-step, 3-seed empirical results do not permit rejecting the null hypothesis at standard significance levels ($\alpha = 0.05$). We treat router steering as a promising inductive prior that warrants verification over extended pre-training horizons ($> 10^4$ steps) and larger seed cohorts ($N \geq 10$).

4 Related Work

Low-Rank and Subspace Optimization: GaLore [6] demonstrated that one-sided gradient projection via periodic SVD enables memory-efficient LLM training. BAdam [7] extended this to block-coordinate updates. CentroidSteer departs from these works by introducing two-sided Cholesky-QR Krylov projections, eliminating SVD stalls, and dynamically steering the projection subspace using router activation geometry.

Matrix Polar and Second-Order Methods: Muon [12] introduced 5th-order Newton-Schulz polynomial iterations to approximate polar decomposition for hidden linear weights. Earlier second-order and preconditioning methods include Shampoo [13,14] and K-FAC [15]. CentroidSteer builds on Muon by restricting polar orthogonalization to 2D attention matrices while establishing specialized low-rank and adaptive update tiers for expert and embedding weights.

Mixture-of-Experts Scaling: Sparse MoE architectures [8–11] drastically expand model capacity while maintaining constant active FLOPs. CentroidSteer repre-

Table 4: **Controlled Routing Steering Ablation on MoE Model (3 Seeds)** under identical initial weights and sequence ordering. Per-seed trajectories and paired statistical tests.

Steering Regime	Seed 42	Seed 1337	Seed 2026	Mean ± Std	Val Loss Drop	Paired Δ vs. Unst
Unsteered Subspace ($\alpha = 0$)	7.7835	7.8767	7.7660	7.8087 ± 0.0595	2.6419 ± 0.0621	—
Random Orthogonal Direction	7.7817	7.7751	7.8616	7.8061 ± 0.0481	2.6445 ± 0.0437	-0.0026 ± 0.098 (p =
Router Centroid Steering	**7.7043**	7.8768	7.8145	$\mathbf{7.7985 \pm 0.0874}$	$\mathbf{2.6521 \pm 0.0863}$	$\mathbf{-0.0102 \pm 0.064}$ (p

sents the first optimizer explicitly co-designed with sparse router activation manifolds.

5 Limitations & Future Work

While CentroidSteer achieves substantial memory savings and solid convergence, several limitations warrant note. First, computing Cholesky-QR basis updates every T_{gap} steps incurs a modest periodic refresh cost (27.57 ms vs. 8.44 ms on non-refresh steps, or 0.957 ms/step amortized at $\tau_{\text{gap}} = 20$ and ≤ 0.096 ms/step at $\tau_{\text{gap}} = 200$), with small boolean host-device readbacks on eager refresh steps to filter unrouted experts and dispatch sliced Householder fallbacks. Second, evaluations in this work comprise 100-step from-scratch ablations ($N = 3$ seeds at 43.5M–90.8M scale) and 200-step checkpoint-continuation runs across two pre-training checkpoint stages ($N = 12$ seeds, $df = 11$, 4 complete 3×3 Latin squares at primary checkpoint step $36,500$ [222.7M tokens] plus supplementary stage-consistency verification across $N = 3$ seeds at step $35,500$ [157.2M tokens] over a $32,768$-token validation set), accompanied by extended-horizon continuation up to 600 steps/seed ($N = 3$, tracking intermediate validation curves at steps 0, 200, 400, and 600, where the paired validation difference narrows from +0.0051 to +0.0031 nats), where full-rank 2D AdamW retains a +0.0037-nat validation advantage ($p = 5.58 \times 10^{-5}$) while remaining equivalent within ± 0.01 nats ($\Delta_{\min} = \pm 0.0047$ nats, $p_{\text{TOST}} = 1.52 \times 10^{-7}$) conditional on the evaluated checkpoints and FineWeb-Edu pool; multi-thousand-step continuation and from-scratch pre-training over billions of tokens will be required to characterize long-horizon scaling laws. Future work will investigate dynamic rank scheduling and distributed tensor-parallel basis synchronization.

6 Conclusion

CentroidSteer demonstrates that low-rank gradient projection and polar orthogonalization can be unified into a cohesive, mathematically grounded optimizer architecture. By guiding expert parameter updates along dynamic router activation manifolds and preserving sensitive embeddings, CentroidSteer reduces persistent expert optimizer memory by 98.4% while providing empirical convergence improvements on modern sparse Mixture-of-Experts models.

References

[1] J. Kaplan et al., "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.

[2] A. Radford et al., "Language models are unsupervised multitask learners," *OpenAI blog*, 2019.

[3] T. Chen et al., "Training deep nets with sublinear memory cost," *arXiv preprint arXiv:1604.06174*, 2016.

[4] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *ICLR*, 2019.

[5] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *ICLR*, 2015.

[6] J. Zhao et al., "GaLore: Memory-efficient LLM training by gradient low-rank projection," in *ICML*, 2024.

[7] Q. Liu et al., "BAdam: A memory-efficient full parameter training method for large language models," in *NeurIPS*, 2024.

[8] N. Shazeer et al., "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *ICLR*, 2017.

[9] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *JMLR*, 2022.

[10] A. Q. Jiang et al., "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.

[11] DeepSeek-AI, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[12] K. Jordan et al., "Muon: An optimizer for hidden layers in neural networks," *Technical Report*, 2024.

[13] V. Gupta, T. Koren, and Y. Singer, "Shampoo: Preconditioned stochastic tensor optimization," in *ICML*, 2018.

[14] R. Anil et al., "Scalable second order optimization for deep learning," *arXiv preprint arXiv:2002.09018*, 2020.

[15] J. Martens and R. Grosse, "Optimizing neural networks with Kronecker-factored approximate curvature," in *ICML*, 2015.